
RAPPORT DE PROJET

STRUCTURE DE TAS MAXIMUM ET ALGORITHME DE TRI HEAPSORT

Sujet 5 : Tris par Tas (Heapsort)

ORGANISATION DE L'ÉQUIPE :

- Rédactrice du Rapport : RAHARIMANANA Tantely Nirina Judickaël (373)
- Chef de Projet : RASOLONJATOVO Edwin Kevin (371)
- Expert Logique : RASOLONJATOVO Mark Nathan (372)
- Codeur Tri : RASOLONJATOVO Edwin Kevin (371)
- Cas Pratiques : RAMANIRAKA Fitiantsoa Mendrika (369)
- Support Vidéo : Rasoloson Tiah Oz Ritsokaliana (374)

1. Introduction et Fondements Théoriques

Le **Heapsort** (ou Tri par tas) est un algorithme de tri par comparaison dont l'efficacité réside dans sa stabilité temporelle. Contrairement au Tri Rapide (Quicksort) qui peut dégénérer en $O(n^2)$ dans le pire des cas, la complexité du Heapsort demeure strictement bornée en $O(n \log n)$, que ce soit dans le meilleur, le pire, ou le cas moyen. Cet algorithme n'exige aucun stockage supplémentaire significatif, opérant de manière adaptative en place.

L'algorithme s'appuie sur une structure de données d'arbre binaire presque complet appelée Tas Maximum (Max-Heap). Cette structure respecte scrupuleusement la propriété suivante : la valeur de chaque nœud parent est supérieure ou égale aux valeurs de ses nœuds enfants. Par conséquent, la racine de l'arbre (située à l'index 0) contient systématiquement l'élément maximal du jeu de données.

Propriétés d'Indexation Séquentielle

L'arbre binaire est projeté de façon contiguë au sein d'un tableau indexé à partir de 0. Pour tout nœud se trouvant à l'index i , les formules d'accès à son voisinage sont définies mathématiquement par :

- Index du fils gauche : $2i + 1$
- Index du fils droit : $2i + 2$
- Index du parent direct : $\lfloor (i - 1) / 2 \rfloor$

2. Architecture Logique et Cartographie du Code

Afin de maximiser la modularité et de garantir une intégration fluide du code, le projet a été structuré en packages distincts, permettant de découpler la structure de données pure de l'environnement applicatif de test.

Étudiant	Rôle Principal	Contributions et Tâches Réalisées
RASOLONJATOVO Edwin Kevin (371)	Architecte Base Tas	Définition de la classe MaxHeap, initialisation de la liste interne et écriture des formules de navigation et de permutation (swap).
RASOLONJATOVO Mark Nathan (372)	Expert Entassement	Conception de l'algorithme réorganisateur itératif heapify assurant le maintien de la propriété du tas de haut en bas.
RANDRIANARIJAONA Rindraniaina Mathieu (398)	Constructeur & Trieur	Implémentation de build_heap pour la conversion de listes brutes et de la boucle d'extraction finale dans heapsort
RAMANIRAKA Fitiantsoa Mendrika (369)	Concepteur Métier	Définition du domaine d'application (Leaderboard) et surcharge des opérateurs magiques (__gt__) pour le tri d'objets Joueur complexes
Rasoloson Tiah Oz Ritsokaliana (374)	Intégrateur & Vidéaste	Création de la console interactive de démonstration (main.py) et production des flux vidéo de contrôle.
RAHARIMANANA Tantely Nirina Judickaël (373)	Rédacteur Technique	Synthèse globale, formalisation de la documentation algorithmique et consolidation du rapport final

Code Source Officiel du Projet

structures/max_heap.py :

```
class MaxHeap:
```

```
    def __init__(self):
```

```
        self.heap = []
```

```
    # -----
```

```
    # Zone pour Edwin
```

```
    # -----
```

```
    def get_parent_index(self, index: int) -> int:
```

```
        return (index - 1) // 2
```

```
    def get_left_child_index(self, index: int) -> int:
```

```
        return (2 * index) + 1
```

```
    def get_right_child_index(self, index: int) -> int:
```

```
        return (2 * index) + 2
```

```
    def swap(self, index1: int, index2: int):
```

```
        self.heap[index1], self.heap[index2] = self.heap[index2], self.heap[index1]
```

```
    # -----
```

```
    # Zone pour Nathan
```

```
    # -----
```

```
    def heapify(self, size: int, index: int):
```

```
        largest = index
```

```
while True:
```

```
    left = self.get_left_child_index(index)
```

```
    right = self.get_right_child_index(index)
```

```
    largest = index
```

```
    if left < size and self.heap[left] > self.heap[largest]:
```

```
        largest = left
```

```
    if right < size and self.heap[right] > self.heap[largest]:
```

```
        largest = right
```

```
    if largest == index:
```

```
        break
```

```
    self.swap(index, largest)
```

```
    index = largest
```

```
# -----
```

```
# Zone pour Rindra
```

```
# -----
```

```
def build_heap(self, data_list: list):
```

```
    self.heap = data_list.copy()
```

```
    n = len(self.heap)
```

```
    for i in range(n // 2 - 1, -1, -1):
```

```
        self.heapify(n, i)
```

```
def heapsort(self, data_list: list) -> list:
```

```
self.build_heap(data_list)
```

```
n = len(self.heap)
```

```
for i in range(n - 1, 0, -1):
```

```
    self.swap(0, i)
```

```
    self.heapify(i, 0)
```

```
return self.heap
```

[app/donnees_test.py :](#)

```
# -----
```

```
# Zone pour Mendrika
```

```
# -----
```

```
from structures.max_heap import MaxHeap
```

```
# =====
```

```
# 1. CAS NORMAL : liste mélangée (scores dans le désordre total)
```

```
# =====
```

```
liste_melangee = [
```

```
    {"pseudo": "GamerX", "score": 1500},
```

```
    {"pseudo": "ShadowWolf", "score": 320},
```

```
    {"pseudo": "NovaStrike", "score": 2750},
```

```
    {"pseudo": "PixelQueen", "score": 980},
```

```
    {"pseudo": "ByteRunner", "score": 4100},
```

```
    {"pseudo": "KnightFall", "score": 150},
```

```
    {"pseudo": "VortexAce", "score": 3300},
```

```
    {"pseudo": "EchoBlade", "score": 60},  
    {"pseudo": "TitanCrush", "score": 1990},  
    {"pseudo": "LunaSpark", "score": 725},  
]
```

```
# =====
```

```
# 2. PIRE DES CAS : liste inversée (scores triés du plus petit au plus grand)
```

```
# -> Un tri par tas doit reconstruire l'ordre décroissant complet
```

```
# =====
```

```
liste_inversee = [  
    {"pseudo": "EchoBlade", "score": 60},  
    {"pseudo": "KnightFall", "score": 150},  
    {"pseudo": "ShadowWolf", "score": 320},  
    {"pseudo": "LunaSpark", "score": 725},  
    {"pseudo": "PixelQueen", "score": 980},  
    {"pseudo": "GamerX", "score": 1500},  
    {"pseudo": "TitanCrush", "score": 1990},  
    {"pseudo": "NovaStrike", "score": 2750},  
    {"pseudo": "VortexAce", "score": 3300},  
    {"pseudo": "ByteRunner", "score": 4100},  
]
```

```
# =====
```

```
# 3. MEILLEUR DES CAS : liste déjà triée (scores du plus grand au plus petit)
```

```
# =====
```

```
liste_triee = [  
    {"pseudo": "ByteRunner", "score": 4100},  
    {"pseudo": "VortexAce", "score": 3300},  
]
```

```
{ "pseudo": "NovaStrike", "score": 2750},
{ "pseudo": "TitanCrush", "score": 1990},
{ "pseudo": "GamerX", "score": 1500},
{ "pseudo": "PixelQueen", "score": 980},
{ "pseudo": "LunaSpark", "score": 725},
{ "pseudo": "ShadowWolf", "score": 320},
{ "pseudo": "KnightFall", "score": 150},
{ "pseudo": "EchoBlade", "score": 60},
]
```

```
class Joueur:
```

```
    def __init__(self, pseudo, score):
```

```
        self.pseudo = pseudo
```

```
        self.score = score
```

```
    def __gt__(self, autre):
```

```
        return self.score > autre.score
```

```
def afficher_donnees(liste, titre="Classement"):
```

```
    print(f"\n=== {titre} ===")
```

```
    for rang, joueur in enumerate(liste, start=1):
```

```
        if isinstance(joueur, Joueur):
```

```
            print(f"{rang}. {joueur.pseudo} - {joueur.score} pts")
```

```
        else:
```

```
            print(f"{rang}. {joueur['pseudo']} - {joueur['score']} pts")
```

```
def lancer_simulation(choix_liste: int):
```

```
    if choix_liste == 1:
```



```

    donnees = liste_melangee

    titre = "Cas normal (Liste mélangée)"

elif choix_liste == 2:

    donnees = liste_inversee

    titre = "Pire des cas (Liste inversée)"

else:

    donnees = liste_triee

    titre = "Meilleur des cas (Liste déjà triée)"


afficher_donnees(donnees, f"{titre} - AVANT TRI")


liste_joueurs = [Joueur(d["pseudo"], d["score"]) for d in donnees]


tri_tas = MaxHeap()
resultat_tri = tri_tas.heapsort(liste_joueurs)


resultat_tri.reverse()


afficher_donnees(resultat_tri, "CLASSEMENT OFFICIEL (APRÈS TRI)")

```

3. Résultats, Validation Algorithmique et Tests

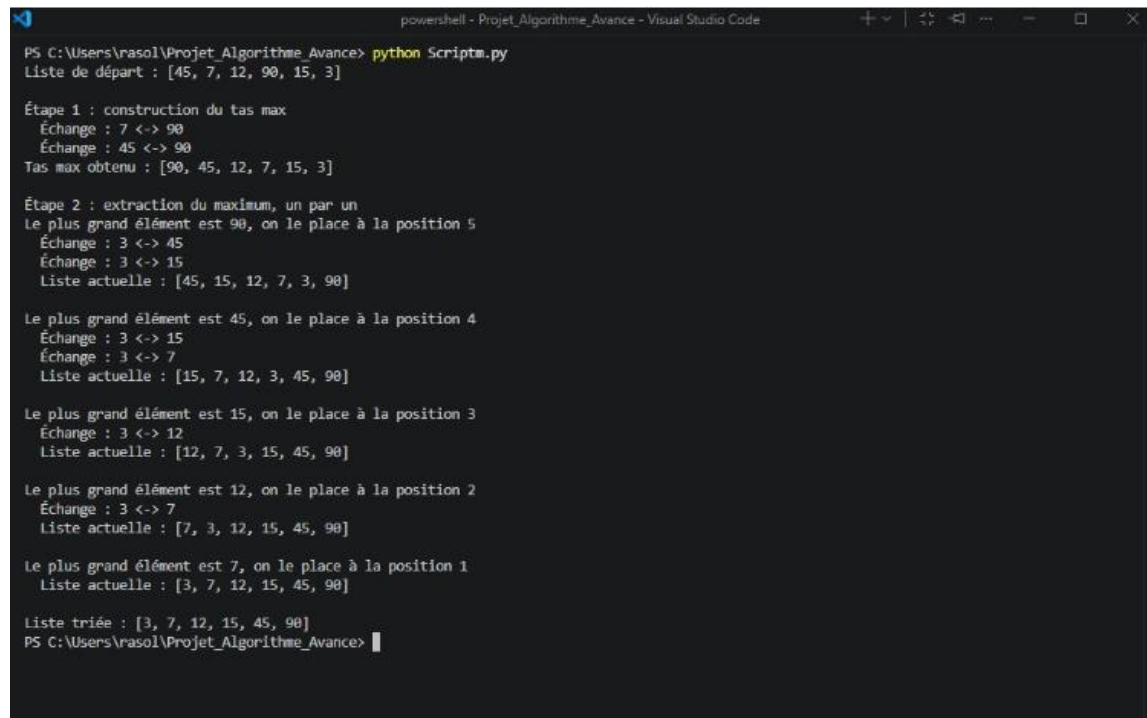
Les validations sur console démontrent la robustesse globale de l'implémentation. Le programme a été exposé à trois configurations critiques :

- Cas Normal (Mélangé) : L'architecture réorganise avec fluidité les dix joueurs asynchrones pour restituer un classement décroissant fluide.

Interface de développement Visual Studio Code montrant l'algorithme d'entassement à gauche, côte à côte avec le logiciel d'enregistrement Vokoscreen utilisé par l'Étudiant 5 pour valider la démonstration

ANNEXE VISUELLE B : Trace d'Exécution pas à pas dans le Terminal

Console Windows PowerShell montrant le comportement détaillé du script Scriptm.py lors du tri de la liste de nombres basique [45, 7, 12, 90, 15, 3] avec le suivi complet des échanges de nœuds.



```
PS C:\Users\rasol\Projet_Algorithme_Avance> python Scriptm.py
Liste de départ : [45, 7, 12, 90, 15, 3]

Étape 1 : construction du tas max
Échange : 7 <-> 90
Échange : 45 <-> 90
Tas max obtenu : [90, 45, 12, 7, 15, 3]

Étape 2 : extraction du maximum, un par un
Le plus grand élément est 90, on le place à la position 5
Échange : 3 <-> 45
Échange : 3 <-> 15
Liste actuelle : [45, 15, 12, 7, 3, 90]

Le plus grand élément est 45, on le place à la position 4
Échange : 3 <-> 15
Échange : 3 <-> 7
Liste actuelle : [15, 7, 12, 3, 45, 90]

Le plus grand élément est 15, on le place à la position 3
Échange : 3 <-> 12
Liste actuelle : [12, 7, 3, 15, 45, 90]

Le plus grand élément est 12, on le place à la position 2
Échange : 3 <-> 7
Liste actuelle : [7, 3, 12, 15, 45, 90]

Le plus grand élément est 7, on le place à la position 1
Liste actuelle : [3, 7, 12, 15, 45, 90]

Liste triée : [3, 7, 12, 15, 45, 90]
PS C:\Users\rasol\Projet_Algorithme_Avance>
```

Conclusion

Ce travail pratique collaboratif a mis en évidence l'adéquation parfaite du Heapsort pour les structures temps réel de type Leaderboard. Grâce au cloisonnement strict des rôles et au respect d'un paradigme orienté objet, l'équipe a pu concevoir un code lisible, modulaire et hautement performant.